

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №4
по дисциплине «Методы тестирования
программного обеспечения»
«Автоматизированное тестирование»
Вариант 1

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Описание средств автоматизации тестирования	6
2.1 JUnit	6
2.1.1 Основные особенности JUnit	6
2.1.2 Этапы написания тестов в JUnit	7
2.2 Selenium WebDriver	7
2.2.1 Возможности Selenium WebDriver	7
2.2.2 Архитектурные компоненты	8
2.2.3 Ключевые понятия	8
2.2.4 Процесс тестирования	8
2.3 Page Object	9
2.4 TestNG	10
2.4.1 Основные особенности TestNG	10
2.4.2 Основные аннотации TestNG	11
2.4.3 Этапы написания тестов в TestNG	11
3 Задача 1	12
3.1 Подготовительные операции	12
3.2 Тесты функции <code>average</code>	12
3.3 Тесты функции <code>countOccurrences</code>	14
3.4 Тесты функции <code>reverse</code>	16
3.5 Тесты функции <code>sum</code>	17
3.6 Общая структура тестов	18
3.7 Результаты тестирования	19
4 Задача 2	21
4.1 Настройка драйвера	24
4.2 NavigationTraining1	25
4.2.1 Метод навигации	25
4.2.2 Метод отладки	25
4.2.3 Основной тест	26
4.3 HomePageContentTests	26
4.3.1 Проверка иконок преимуществ (Benefit Icons)	27
4.3.2 Проверка текстов под иконками	27
4.3.3 Проверка <code>iframe</code> с кнопкой	27
4.3.4 Проверка левого меню (Sidebar)	27

4.4 Результаты тестирования	28
Заключение	29
Список использованной литературы	30

Введение

Автоматизированное тестирование - это метод проверки программного обеспечения с использованием специальных инструментов и скриптов. В отличие от ручного тестирования, тестовые примеры разрабатываются однократно и многократно выполняются автоматически, а полученные результаты сравниваются с ожидаемыми.

Автоматизация тестирования значительно ускоряет процесс проверки приложений, сокращая время тестирования за счет делегирования повторяющихся действий программным средствам. Также оно минимизирует человеческий фактор, снижая риск пропуска дефектов и ошибок.

С помощью автоматизации можно создавать стандартизированные тесты, которые гарантируют одинаковое прохождение каждого сценария. Это способствует объективности оценки результатов и высокий уровень качества.

В то же время, у автоматизированного тестирования есть ограничения. Оно не позволяет оценить удобство использования интерфейса, поскольку не учитывает визуальное восприятие и субъективный опыт пользователя. Автоматические тесты проверяют заранее заданные сценарии, поэтому они не способны выявить неожиданные дефекты, возникающие при нестандартных действиях пользователя.

Таким образом, автоматизированное тестирование не является полной заменой ручному тестированию, а дополняет его для более эффективной работы. Автоматизация оптимальна для рутинных и повторяющихся задач, в то время как ручное тестирование необходимо для сложных сценариев и оценки пользовательского опыта.

1 Постановка задачи

Цель работы — изучить и применить инструменты автоматизированного тестирования: фреймворк `JUnit` для модульного тестирования и `Selenium WebDriver` вместе с `TestNG` для UI-тестирования веб-приложений.

В ходе выполнения данной лабораторной работы требуется выполнить следующие задания:

1. Создать юнит-тесты для библиотеки `lab-1.0.jar` с использованием фреймворка `JUnit`:
 - Разработать юнит-тесты для четырех методов класса `ArrayUtils.java`.
 - Запустить тесты через используемую среду разработки.
 - Продемонстрировать результаты выполнения тестов.
2. Провести тестирование веб-сайта с использованием `Selenium WebDriver` и `TestNG`:
 - Написать два теста, проверяющих корректное отображение и работу страниц веб-сайта.
 - Организовать код тестов в соответствии с `Java Code Convention`.
 - Запустить тесты через конфигурационный файл `TestNG suite.xml`.

2 Описание средств автоматизации тестирования

2.1 JUnit

JUnit — это мощный фреймворк для модульного тестирования Java-приложений. Он предоставляет разработчикам удобные инструменты для создания, организации и выполнения автоматизированных тестов, в том числе тестирование отдельных модулей. Это способствует повышению качества кода и ускорению процесса разработки.

2.1.1 Основные особенности JUnit

- JUnit использует систему аннотаций для управления жизненным циклом тестов, что упрощает написание тестов и делает код более читабельным. Основные аннотации:
 - `@Test` — обозначает метод как тестовый случай;
 - `@BeforeEach` — метод выполняется перед каждым тестом;
 - `@AfterEach` — метод выполняется после каждого теста;
 - `@BeforeAll` — статический метод, выполняемый один раз перед всеми тестами;
 - `@AfterAll` — статический метод, выполняемый один раз после всех тестов.
- JUnit предоставляет различные методы ассертов для проверки ожидаемых результатов тестов:
 - `assertEquals(expected, actual)` — проверяет равенство двух значений;
 - `assertTrue(condition)` — проверяет, что условие истинно;
 - `assertFalse(condition)` — проверяет, что условие ложно;
 - `assertNotNull(object)` — проверяет, что объект не является null;
 - `assertNull(object)` — проверяет, что объект является null;
 - `assertThrows(exception, executable)` — проверяет генерацию исключения.

- JUnit отличается интуитивно понятным API, минимальными требованиями к настройке и прозрачной моделью выполнения тестов.
- JUnit совместим с разными редакторами кода. Он поддерживает интеграцию с IntelliJ IDEA, Eclipse, NetBeans, Maven, Gradle и CI-системами (Jenkins, Travis CI).

2.1.2 Этапы написания тестов в JUnit

1. Реализация теста: создание тестового класса и написание тестового метода с аннотацией `@Test`.
2. Настройка и очистка: использование аннотаций `@BeforeEach` и `@AfterEach` для выполнения операций перед и после теста.
3. Запуск тестов: использование встроенных средств IDE или командной строки (`mvn test`, `gradle test`) для выполнения тестов.
4. Просмотр отчётов о пройденных и неудачных тестах.

2.2 Selenium WebDriver

Selenium WebDriver — это фреймворк с открытым исходным кодом для автоматизации тестирования веб-приложений, представляющий собой основной продукт комплекта Selenium. Он предоставляет программный интерфейс для взаимодействия с браузерами, позволяя эмулировать действия пользователя на веб-страницах.

2.2.1 Возможности Selenium WebDriver

Selenium WebDriver позволяет:

- Автоматизировать работу с реальным браузером (локально или удалённо);
- Максимально точно имитировать действия пользователя;
- Тестировать в различных браузерах (Chrome, Firefox, Edge, Safari, Opera);
- Интегрироваться с фреймворками тестирования (JUnit, TestNG);
- Поддерживать параллельное выполнение тестов;
- Выполнять кросс-браузерное и кросс-платформенное тестирование (Windows, MacOS, Linux).

2.2.2 Архитектурные компоненты

Для работы с WebDriver требуется три основных компонента:

1. Браузер: реальный браузер конкретной версии, установленный на определённой ОС с индивидуальными настройками.
2. Драйвер браузера: веб-сервер, который запускает браузер, отправляет ему команды и закрывает браузер по завершении. Для каждого браузера существует свой драйвер (ChromeDriver, GeckoDriver для Firefox и др.).
3. Тест: набор команд на языке программирования (Java, Python, C#, Ruby, JavaScript), использующий библиотеки Selenium WebDriver bindings.

2.2.3 Ключевые понятия

Основными сущностями в Selenium WebDriver являются:

- WebDriver — сущность, ответственная за управление браузером. Основной ход теста строится именно вокруг экземпляра этой сущности. Пример создания:

```
1 WebDriver driver = new ChromeDriver();
```

- WebElement представляет собой абстракцию над веб-элементом (кнопка, ссылка, поле ввода и т.п.). WebElement инкапсулирует методы для взаимодействия пользователя с элементами и получения их текущего статуса:

```
1 WebElement searchField = driver.findElement(By.name("q"));
2 searchField.sendKeys("Selenium WebDriver");
3 searchField.submit();
```

- Ву (локаторы) - абстракция над локатором веб-элемента. Этот класс инкапсулирует информацию о селекторе, а также сам локатор элемента.

2.2.4 Процесс тестирования

Принцип выполнения команд в Selenium WebDriver:

1. Команда из тестового скрипта преобразуется в HTTP-запрос через JSON Wire Protocol.

2. Запрос передается соответствующему драйверу браузера.
3. Драйвер интерпретирует запрос и выполняет необходимые действия в браузере.
4. Результат действия возвращается обратно в виде HTTP-ответа.
5. Ответ преобразуется в формат, понятный тестовому скрипту.

Этапы работы с WebDriver

1. Создание экземпляра WebDriver и открытие браузера.
2. Переход к нужной веб-странице с помощью метода `get()`.
3. Обнаружение элементов на странице с использованием различных локаторов.
4. Выполнение действий над элементами (клик, ввод текста, выбор из выпадающих списков).
5. Получение информации о состоянии элементов и проверка результатов.
6. Закрытие браузера и освобождение ресурсов.

2.3 Page Object

Page Object — шаблон проектирования для автоматизации тестирования веб-приложений. Он помогает инкапсулировать работу с отдельными элементами страницы, что позволяет использовать одни и те же объекты страниц в различных тестах.

Основные принципы Page Object

1. Одна страница — один класс. Каждая веб-страница представляется отдельным классом.
2. Локаторы хранятся внутри класса, методы описывают действия пользователя.
3. Тесты работают только через методы Page Object, не обращаясь напрямую к элементам страницы.

Основные преимущества использования паттерна Page Object:

- разделение кода тестов и кода описания страниц;
- объединение всех действий по работе с веб-страницей в одном месте.

Паттерн Page Object в Selenium реализован с помощью библиотеки PageFactory и класса страницы. Page Object представляет собой отдельный класс, содержащий локаторы элементов, методы для работы с ними и конструктор принимающий в качестве параметра объект WebDriver. Методы класса Page Object могут возвращать объекты других Page Object классов.

2.4 TestNG

TestNG (Test Next Generation) — это современный фреймворк для тестирования Java-приложений, предлагающий более мощные возможности, чем JUnit. Он поддерживает различные виды тестирования: модульное, функциональное, интеграционное и End-to-end тестирование.

2.4.1 Основные особенности TestNG

- TestNG использует расширенную систему аннотаций для управления жизненным циклом тестов.
- Тесты можно настраивать с помощью аннотаций Java и XML-файлов конфигурации.
- Тесты можно объединять в группы с помощью аннотации `@Test(groups="groupName")`, что позволяет запускать только определённые группы тестов.
- Параллельное выполнение: TestNG поддерживает параллельное выполнение тестов на уровне методов, классов и тестовых наборов.
- Поддержка параметризованных тестов. Аннотации `@Parameters` и `@DataProvider` позволяют передавать параметры в тесты, что упрощает тестирование с различными входными данными.
- Встроенная поддержка многопоточности для нагрузочного тестирования.

2.4.2 Основные аннотации TestNG

- `@Test` — основная аннотация для тестовых методов;
- `@BeforeSuite/@AfterSuite` — выполняются до/после всего набора тестов;
- `@BeforeTest/@AfterTest` — выполняются до/после тестового блока в XML;
- `@BeforeClass/@AfterClass` — выполняются до/после методов класса;
- `@BeforeMethod/@AfterMethod` — выполняются до/после каждого тестового метода;
- `@BeforeGroups/@AfterGroups` — выполняются до/после группы тестов;
- `@Parameters` — для параметризации тестов через XML;
- `@DataProvider` — для подачи тестовых данных через код.

2.4.3 Этапы написания тестов в TestNG

1. Создание тестового класса, написание тестовых методов с аннотацией `@Test`.
2. Настройка пред- и пост-условий: использование аннотаций `@BeforeSuite`, `@AfterSuite`, `@BeforeTest`, `@AfterTest`, `@BeforeMethod`, `@AfterMethod` для настройки и очистки.
3. Конфигурация тестов: Добавление информации о тесте (имя класса, группы, параметры) через XML-файл или аннотации.
4. Запуск тестов через IDE (IntelliJ IDEA, Eclipse), Maven (`mvn test`) или XML-конфигурацию для управления тестовыми наборами.
5. Анализ результатов: просмотр отчётов TestNG (HTML-отчёты), которые включают информацию о пройденных и неудачных тестах, времени выполнения и других метриках.

3 Задача 1

Дано: библиотечный файл `lab-1.0.jar`, содержащий класс `ru.spbstu.hsai.ArrayUtils`, предназначенный для обработки массивов целых чисел.

Требуется:

- Создать новый Java-проект.
- Добавить в зависимости проекта фреймворк JUnit 5 и библиотеку `lab-1.0.jar`.
- Выбрать четыре метода из класса библиотеки `ArrayUtils` и написать для них юнит-тесты с использованием фреймворка JUnit 5. Необходимо придерживаться следующей организации кода:
 - Все подготовительные операции должны быть вынесены в методы с аннотацией `@Before`.
 - Все завершающие операции должны быть вынесены в методы с аннотациями `@After`.
 - Данные для тестов должны передаваться через параметры с аннотациями `@ParametrizedTest` и `@ValueSource`.
- Запустить тесты через интегрированную среду разработки (IDE).

3.1 Подготовительные операции

Класс `ArrayUtilsTest` представляет абстрактный базовый класс, от которого наследуются тестовые классы, проверяющие конкретный метод библиотеки. В классе `ArrayUtilsTest` определен метод `setUp()` с выводом сообщения о начале тестирования. Также для сравнения чисел с плавающей точкой введена константа `DELTA = 1e-4`. Код класса `ArrayUtilsTest` приведен в листинге 1.

3.2 Тесты функции `average`

Дано: статическая функция `average` класса `ArrayUtils`, предназначенная для вычисления среднего арифметического элементов массива. Функция принимает массив целых чисел `int[] array` и возвращает среднее арифметическое типа `double`. В случае массива с нулевой длиной выбрасывает `IllegalArgumentException`.

Можно составить тесты согласно спецификации, используя техники классов эквивалентностей и анализа граничных значений.

Разбиение на классы эквивалентности представлено в таблице 1.

Таблица 1. Классы эквивалентности

Входные данные	Допустимые классы	Недопустимые классы
int[] array	array - массив целых чисел (1)	array - null массив (2)
		array - пустой массив (3)

В таблице 2 представлены результаты тестов, покрывающих все классы эквивалентности.

Таблица 2. Тесты, покрывающие все классы эквивалентности

№	array	Ожидаемый результат	Фактический результат	Результат тестирования
1	{5, 7, 23, -2, 4}	7.4	7.4	Пройден
2	null	IllegalArgumentException	IllegalArgumentException	Пройден
3	{ }	IllegalArgumentException	IllegalArgumentException	Пройден

Граничные значения:

- Верхние: элемент массива $\text{array}[i] = 2147483647$ (Integer.MAX_VALUE)
- Нижние: элемент массива $\text{array}[i] = -2147483648$ (Integer.MIN_VALUE)

Таблица 3. Проверка граничных значений

№	array	Ожидаемый результат	Фактический результат	Результат тестирования
1	{2147483647}	(double)Integer.MAX_VALUE	(double)Integer.MAX_VALUE	Пройден
2	{-2147483647 }	(double)Integer.MIN_VALUE	(double)Integer.MIN_VALUE	Пройден

Код класса `AverageTest` приведен в листинге 2.

Выводы: все тесты для функции `average` проходят успешно, следовательно, она корректно реализована.

3.3 Тесты функции `countOccurrences`

Дано: статическая функция `countOccurrences` класса `ArrayUtils`, предназначенная для вычисления количества элементов, равных заданному значению. Функция принимает массив целых чисел `int[] array` и искомое значение `int value`. Возвращает количество вхождений значения в массив типа `int`. В случае массива с нулевой длиной выбрасывает `IllegalArgumentException`.

Можно составить тесты согласно спецификации, используя техники классов эквивалентностей и анализа граничных значений.

Разбиение на классы эквивалентности представлено в таблице 4.

Таблица 4. Классы эквивалентности

Входные данные	Допустимые классы	Недопустимые классы
<code>int[] array</code>	<code>array</code> - массив целых чисел	<code>array</code> - null массив
<code>int value</code>	<code>value</code> - целое число	-

В таблице 5 представлены результаты тестов, покрывающих все классы эквивалентности.

Таблица 5. Тесты, покрывающие все классы эквивалентности

№	array, value	Ожидаемый результат	Фактический результат	Результат тестирования
1	{7}, 7	1	0	Не пройден
2	{1, 2, 3}, 1	1	0	Не пройден
3	{1, 1, 2}, 1	2	1	Не пройден
4	{1, 2, 1}, 1	2	1	Не пройден
5	{2, 3, 1}, 1	1	1	Пройден
6	{2, 3, 3, 3}, 3	3	3	Пройден
7	{1, 2, 3}, 99	0	0	Пройден
8	{}, 5	0	0	Пройден
9	null, 5	IllegalArgumentException	IllegalArgumentException	Пройден
10	null, 0	IllegalArgumentException	IllegalArgumentException	Пройден

Граничные значения:

- Верхние: элемент массива `array[i] = 2147483647` (`Integer.MAX_VALUE`)
- Нижние: элемент массива `array[i] = -2147483648` (`Integer.MIN_VALUE`)

Таблица 6. Проверка граничных значений

№	array	Ожидаемый результат	Фактический результат	Результат тестирования
1	{0, 2147483647, 2147483647}, 2147483647	2	2	Пройден
2	{0, -2147483647, -2147483647}, -2147483647	2	2	Пройден

Код класса `CountOccurrencesTest` приведен в листинге 3.

Выводы: тесты, где искомое значение находится на первом месте, выдают ошибку. Это связано с тем, что цикл подсчета начинается с индекса 1, пропуская первый элемент массива. Это является дефектом исходной библиотеки.

3.4 Тесты функции reverse

Дано: статическая функция `reverse` класса `ArrayUtils`, предназначенная для возврата массива в обратном порядке. Функция принимает массив целых чисел `int[] array` и возвращает новый массив с элементами в обратном порядке. В случае массива с нулевой длиной выбрасывает `IllegalArgumentException`.

Можно составить тесты согласно спецификации, используя техники классов эквивалентностей и анализа граничных значений.

Разбиение на классы эквивалентности представлено в таблице 7.

Таблица 7. Классы эквивалентности

Входные данные	Допустимые классы	Недопустимые классы
<code>int[] array</code>	<code>array</code> - массив целых чисел	<code>array</code> - null массив

В таблице 8 представлены результаты тестов, покрывающих все классы эквивалентности.

Таблица 8. Тесты, покрывающие все классы эквивалентности

№	<code>array, value</code>	Ожидаемый результат	Фактический результат	Результат тестирования
1	<code>{5}</code>	<code>{5}</code>	<code>{5}</code>	Пройден
2	<code>{1, -2, 3, 0, -5}</code>	<code>{-5, 0, 3, -2, 1}</code>	<code>{1, 0, 3, -2, -5}</code>	Не пройден
4	<code>{}</code>	<code>{}</code>	<code>{}</code>	Пройден
5	<code>null</code>	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	Пройден

Граничные значения:

- Верхние: элемент массива `array[i] = 2147483647` (`Integer.MAX_VALUE`)
- Нижние: элемент массива `array[i] = -2147483648` (`Integer.MIN_VALUE`)

Таблица 9. Проверка граничных значений

№	array	Ожидаемый результат	Фактический результат	Результат тестирования
1	{2147483647}	{2147483647}	{2147483647}	Пройден
2	{-2147483647}	{-2147483647}	{-2147483647}	Пройден

Код класса `ReverseTest` приведен в листинге 4.

Выводы: тесты, в которых длина массива больше одного и все элементы разные, завершились с ошибкой. Это связано с тем, что после корректного разворота массива метод дополнительно меняет местами первый и последний элементы, в результате получается неверный обратный порядок массива. Это является дефектом исходной библиотеки.

3.5 Тесты функции `sum`

Дано: статическая функция `sum` класса `ArrayUtils`, предназначенная для вычисления суммы всех элементов массива. Функция принимает массив целых чисел `int[] array` и возвращает сумму всех элементов массива типа `int`. В случае массива с нулевой длиной выбрасывает `IllegalArgumentException`.

Можно составить тесты согласно спецификации, используя техники классов эквивалентностей и анализа граничных значений.

Разбиение на классы эквивалентности представлено в таблице 10.

Таблица 10. Классы эквивалентности

Входные данные	Допустимые классы	Недопустимые классы
<code>int[] array</code>	<code>array</code> - массив целых чисел	<code>array</code> - null массив

В таблице 11 представлены результаты тестов, покрывающих все классы эквивалентности.

Таблица 11. Тесты, покрывающие все классы эквивалентности

№	array	Ожидаемый результат	Фактический результат	Результат тестирования
1	{5}	5	5	Пройден
2	{1, -2, 3, 0, -5, 3, 1}	1	1	Пройден
3	{}	0	0	Пройден
4	null	IllegalArgumentException	IllegalArgumentException	Пройден

Граничные значения:

- Верхние: элемент массива `array[i] = 2147483647 (Integer.MAX_VALUE)`
- Нижние: элемент массива `array[i] = -2147483648 (Integer.MIN_VALUE)`

Таблица 12. Проверка граничных значений

№	array	Ожидаемый результат	Фактический результат	Результат тестирования
1	{2147483647, -1}	{2147483646}	{2147483646}	Пройден
2	{-2147483647, 1}	{-2147483646}	{-2147483646}	Пройден

Код класса **SumTest** приведен в листинге 5.

Выводы: все тесты для функции **sum** проходят успешно, следовательно, она корректно реализована.

3.6 Общая структура тестов

Тестовые классы наследуются от базового класса **ArrayUtilsTest**, который предоставляет константу **DELTA** для сравнения вещественных чисел и метод **@BeforeAll** для инициализации.

Для тестирования методов библиотеки **ArrayUtils** используются параметризованные тесты фреймворка JUnit 5 с аннотациями **@ParameterizedTest** в сочетании с **@MethodSource** для сложных входных данных (массивов). Такой подход позволяет корректно обрабатывать массивы и легко расширять наборы тестовых данных без дублирования кода.

3.7 Результаты тестирования

С помощью фреймворка JUnit было разработано 28 тестов для библиотеки `lab-1.0.jar`. Эти тесты проверяют четыре функции библиотеки: `average`, `countOccurrences`, `reverse`, `sum`. Из них:

Для метода `average` разработано 5 тестов, все пройдены успешно.

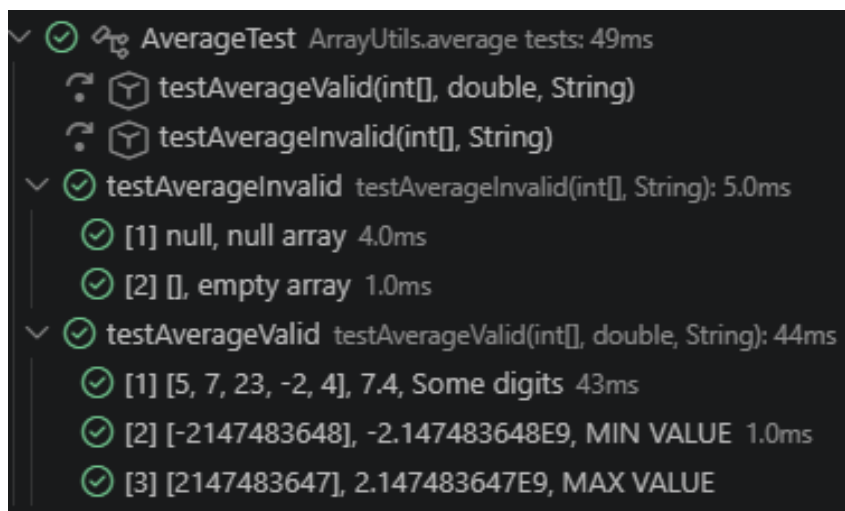


Рис. 1. Результат запуска тестов класса `AverageTest`

Для метода `countOccurrences` разработано 12 тестов, из них 4 не пройдено.

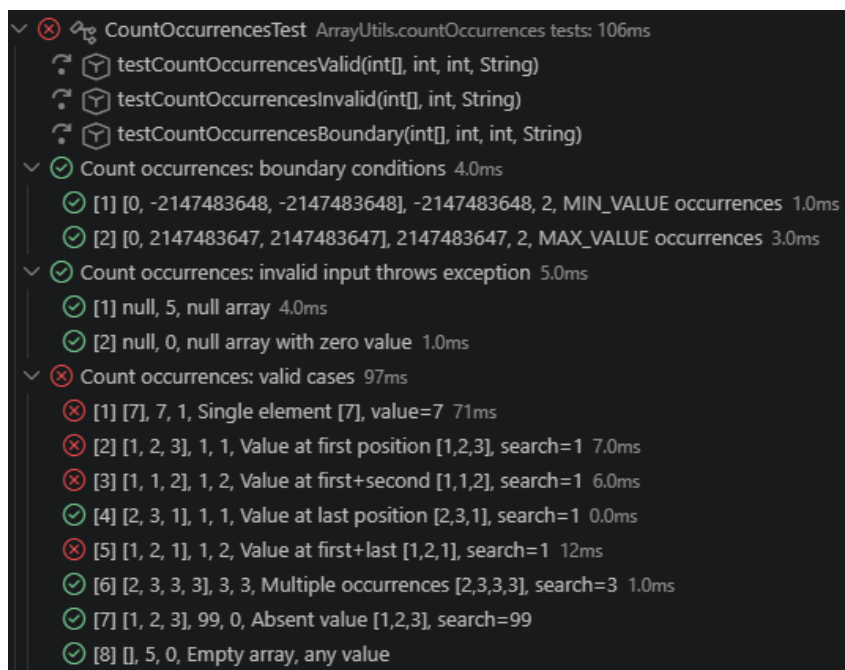


Рис. 2. Результат запуска тестов класса `CountOccurrencesTest`

Для метода `reverse` разработано 6 тестов, из них 1 не пройден.

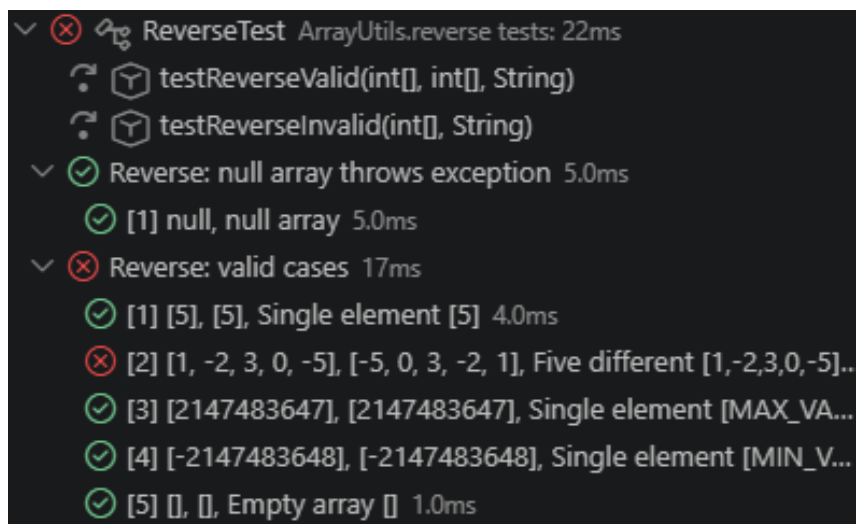


Рис. 3. Результат запуска тестов класса ReverseTest

Для метода `sum` разработано 5 тестов, все пройдены успешно.

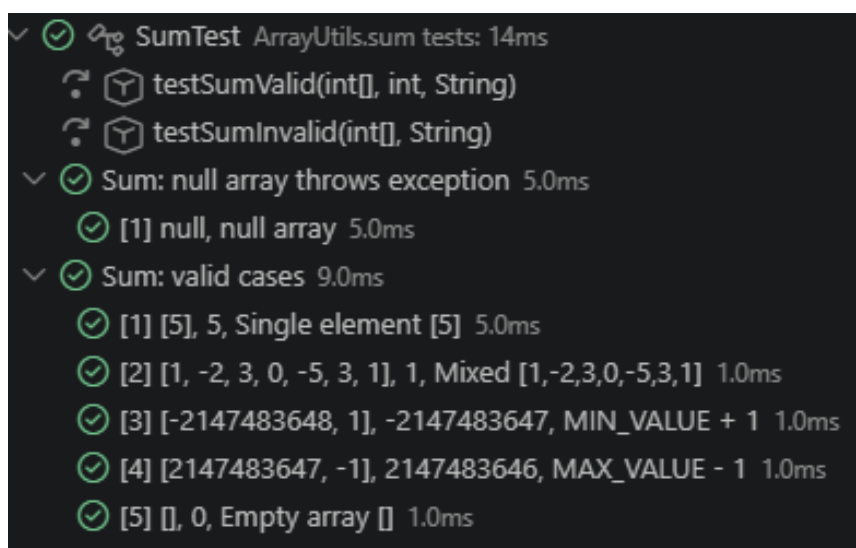


Рис. 4. Результат запуска тестов класса SumTest

4 Задача 2

Дано:

- Веб-сайт для тестирования:
<https://jdi-testing.github.io/jdi-light/index.html>.
- Браузер Google Chrome.
- Фреймворки: Selenium WebDriver, TestNG.

Требуется:

- Подключить зависимости Selenium WebDriver и TestNG в проект.
- Настроить класс инициализации драйвера DriverSetup.
- Создать два теста на языке Java с использованием Selenium WebDriver и фреймворка TestNG для проверки корректности отображения страниц заданного сайта.
- Тесты должны соответствовать стандартам Java Code Convention и запускаться с помощью TestNG suite xml.

Первый и второй тест должны проигрывать 1 и 2 тестовый сценарий соответственно. Первый тестовый сценарий представлен на рисунке 5, второй - на рисунках 6- 7.

Тест 1: testSupportPageFullCheck

#	Шаг	Данные	Ожидаемый результат
1	Перейти на страницу "Support" через главное меню	Пункт меню: "Service" → "Support"	Страница Support открыта
2	Проверить заголовок страницы	Ожидаемый текст: "support" (без учёта регистра)	Заголовок содержит слово "support"
3	Найти форму на странице	Поиск по тегу: <code>form</code>	Найдена хотя бы одна форма
4	Если форма не найдена, найти контактный блок	Поиск по селектору: <code>div[class*='contact'], section[class*='contact']</code>	Найден хотя бы один контактный блок
5	Проверить, что форма отображается	Проверка видимости элемента	Форма видима пользователю
6	Проверить наличие input-полей в форме	Поиск по тегу: <code>input</code> внутри формы	В форме есть хотя бы 1 input
7	Проверить наличие textarea в форме	Поиск по тегу: <code>textarea</code> внутри формы	В форме есть хотя бы 1 textarea
8	Проверить наличие кнопки отправки в форме	Поиск по селектору: <code>button, input[type='submit']</code>	В форме есть кнопка отправки

Рис. 5. Первый тестовый сценарий

Тестовый сценарий: HomePageContentTests

Тест 1: testBenefitIconsCount

#	Шаг	Данные	Ожидаемый результат
1	Найти иконки преимуществ	Селектор: <code>.benefit-icon span, .benefit-icon img, .benefit-item img</code>	Элементы найдены
2	Подсчитать количество	Ожидаемое: 4	Количество равно 4

Тест 2: testBenefitIconsVisibility

#	Шаг	Данные	Ожидаемый результат
1	Найти иконки преимуществ	Селектор: <code>.benefit-icon span, .benefit-icon img, .benefit-item img</code>	Элементы найдены
2	Проверить видимость каждой	Метод: <code>isDisplayed()</code> для каждого из 4 элементов	Все 4 иконки отображаются

Тест 3: testBenefitIconsAltAttributes

#	Шаг	Данные	Ожидаемый результат
1	Найти <code>img</code> внутри блоков преимуществ	Селектор: <code>.benefit-item img</code>	Найдено 4 элемента
2	Получить <code>alt</code> для иконки #1	Ожидаемое: содержит "Customization"	<code>alt</code> не пустой, содержит ключевое слово
3	Получить <code>alt</code> для иконки #2	Ожидаемое: содержит "Interface"	<code>alt</code> не пустой, содержит ключевое слово
4	Получить <code>alt</code> для иконки #3	Ожидаемое: содержит "Platform"	<code>alt</code> не пустой, содержит ключевое слово
5	Получить <code>alt</code> для иконки #4	Ожидаемое: содержит "Base"	<code>alt</code> не пустой, содержит ключевое слово

Рис. 6. Второй тестовый сценарий (тесты 1-3)

Тест 4: testBenefitTextsCount

#	Шаг	Данные	Ожидаемый результат
1	Найти текстовые блоки	Класс: benefit-txt	Найдено 4 элемента
2	Проверить количество	Ожидаемое: 4	Количество равно 4

Тест 5: testBenefitTextsContent

#	Шаг	Данные	Ожидаемый результат
1	Найти текстовые блоки	Класс: benefit-txt	Найдено 4 элемента
2	Проверить текст #1	Ожидаемый: To include good practices\and ideas from successful\EPAM project	Текст совпадает, содержит "EPAM"
3	Проверить текст #2	Ожидаемый: To be flexible and\ncustomizable	Текст совпадает, содержит "flexible"
4	Проверить текст #3	Ожидаемый: To be multiplatform	Текст совпадает, содержит "multiplatform"
5	Проверить текст #4	Ожидаемый: Already have good base\n(about 20 internal and\some external projects),\nwish to get more...	Текст совпадает

Тест 6: testBenefitTextsClasses

#	Шаг	Данные	Ожидаемый результат
1	Найти текстовые блоки	Класс: benefit-txt	Найдено 4 элемента
2	Проверить class атрибут	Ожидаемое: содержит "benefit-txt"	Атрибут не пустой, содержит ожидаемый класс
3	Проверить CSS display	Ожидаемое: не равно "none"	Элемент не скрыт

Тест 7: testFrameExistence

#	Шаг	Данные	Ожидаемый результат
1	Найти iframe	ID: frame	iframe найден
2	Проверить видимость	Метод: isDisplayed()	iframe отображается
3	Проверить атрибут src	Ожидаемое: содержит "jdi-testing"	src не пустой, содержит ожидаемый паттерн

Тест 8: testFrameContent

#	Шаг	Данные	Ожидаемый результат
1	Переключиться на iframe	Метод: driver.switchTo().frame("frame")	Контекст переключён
2	Найти кнопку в iframe	ID: frame-button	Кнопка найдена
3	Проверить видимость кнопки	Метод: isDisplayed()	Кнопка отображается
4	Проверить текст кнопки	Ожидаемый: Frame Button	Текст совпадает
5	Проверить активность кнопки	Метод: isEnabled()	Кнопка активна
6	Вернуться к основному контенту	Метод: driver.switchTo().defaultContent()	Контекст возвращён

Тест 9: testLeftMenuCount

#	Шаг	Данные	Ожидаемый результат
1	Найти пункты левого меню	Селектор: ul.sidebar-menu.left > li	Найдено 5 элементов
2	Проверить количество	Ожидаемое: 5	Количество равно 5

Тест 10: testLeftMenuTexts

#	Шаг	Данные	Ожидаемый результат
1	Найти пункты левого меню	Селектор: ul.sidebar-menu.left > li	Найдено 5 элементов
2	Проверить текст #1	Ожидаемый: Home	Текст совпадает
3	Проверить текст #2	Ожидаемый: Contact form	Текст совпадает
4	Проверить текст #3	Ожидаемый: Service	Текст совпадает
5	Проверить текст #4	Ожидаемый: Metals & Colors	Текст совпадает
6	Проверить текст #5	Ожидаемый: Elements packs	Текст совпадает

Рис. 7. Второй тестовый сценарий (тесты 4-10)

4.1 Настройка драйвера

Для инициализации и настройки драйвера WebDriver был создан класс DriverSetup. Основные задачи класса:

- Инициализация драйвера для браузера Chrome с заданными параметрами.
- Выполнение авторизации на тестируемом веб-сайте.
- Корректное освобождение ресурсов после выполнения всех тестов.

Структура класса:

Метод `setup()` предназначен для предварительной настройки окружения перед запуском тестов. Благодаря аннотации `@BeforeTest` он выполняется один раз перед всеми тестами. Он выполняет следующие действия:

1. Создает объект `ChromeOptions` для настройки параметров браузера.
2. Инициализирует драйвер.
3. Открывает тестовую страницу в браузере.
4. Авторизуется через пользовательский интерфейс.

```
1 @BeforeTest
2 public static void setup() {
3     ChromeOptions options = new ChromeOptions();
4     options.addArguments("--start-maximized");
5     driver = new ChromeDriver(options);
6     driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
7     driver.navigate().to("https://jdi-testing.github.io/jdi-light/index.html");
8     driver.findElement(By.cssSelector("html > body > header > div > nav > ul.uui-navigation.navbar-nav.navbar-right > li > a > span")).click();
9     driver.findElement(By.id("name")).sendKeys("Roman");
10    driver.findElement(By.id("password")).sendKeys("Jdi1234");
11    driver.findElement(By.id("login-button")).click();
12 }
```

Метод `exit()` предназначен для освобождения ресурсов. Он корректно закрывает браузер с помощью метода `driver.close()`. Аннотация `@AfterTest` показывает, что он выполняется после завершения всех тестов.


```

1  @AfterTest
2  public static void exit() {
3      driver.close();
4  }
5  }

```

4.2 NavigationTraining1

Класс `NavigationTraining1` наследуется от `DriverSetup` и содержит автоматизированные тесты для проверки функциональности веб-интерфейса сайта <https://jdi-testing.github.io/jdi-light/>. Код класса представлен в листинге 6. Основные особенности:

- Использование `SoftAssert` для накопления результатов всех проверок и успешного завершения даже в случае падения одного из тестов.
- Отладочный метод сохраняет HTML-исходник при возникновении ошибок.
- Навигация происходит через меню с использованием явных ожиданий `WebDriverWait`.

4.2.1 Метод навигации

Метод `navigateTo` обеспечивает переход к целевой странице через главное меню сайта. Для ожидания появления элементов создается экземпляр `WebDriverWait`, выполняется поиск пункта `Service`, наведение на него и клик. Используется XPath-селектор с функцией `normalize-space()` для устойчивости к лишним пробелам в тексте, а также проверка наличия класса `dropdown-toggle` для точного определения элемента выпадающего меню. Внутри открывшегося подменю ищется пункт `Support`, по нему выполняется клик и переход на целевую страницу. Успешность загрузки подтверждает проверка изменения URL.

4.2.2 Метод отладки

Метод предназначен для сохранения диагностической информации при возникновении ошибок в тесте. Для этого он сохраняет скриншот текущего состояния браузера и HTML-исходник страницы.

4.2.3 Основной тест

Тест выполняет комплексную проверку страницы поддержки согласно следующему сценарию:

1. Переход на страницу через меню (вызов `navigateTo("Support")`);
2. Проверка заголовка страницы на наличие слова "support" (без учёта регистра);
3. Поиск формы: сначала по тегу `<form>`, при отсутствии — по селектору `div[class*='contact']`, `section[class*='contact']`;
4. Проверка видимости найденного элемента;
5. Проверка наличия хотя бы одного поля `<input>` внутри формы;
6. Проверка наличия хотя бы одного элемента `<textarea>`;
7. Проверка наличия кнопки отправки (`<button>` или `input[type='submit']`).

Все проверки выполняются через объект `SoftAssert`, что позволяет продолжить выполнение теста даже при неудаче одной из проверок. Финальный вызов `softAssert.assertAll()` агрегирует все накопленные ошибки и выбрасывает исключение при их наличии.

4.3 HomePageContentTests

Класс `HomePageContentTests` наследуется от `DriverSetup` и содержит автоматизированные тесты с использованием `Selenium WebDriver` и `testNG`. Он реализует набор тестов для проверки содержимого главной страницы веб-сайта. Код класса представлен в листинге 7. Основные особенности:

- Использование аннотаций `@Test` с параметрами `priority` и `dependsOnMethods` для управления порядком выполнения и создания зависимостей между тестами;
- Применение различных стратегий поиска элементов: по CSS-селекторам, классам, тегам и атрибутам;
- Проверка атрибутов доступности (`alt`) для соответствия стандартам WCAG;
- Работа с `iframe`: переключение контекста и возврат к основному содержимому.

4.3.1 Проверка иконок преимуществ (Benefit Icons)

- `testBenefitIconsCount` проверяет наличие ровно 4 иконок с помощью CSS-селектора `".benefit-icon span, .benefit-icon img, .benefit-item img"`.
- `testBenefitIconsVisibility` проверяет их видимость методом `isDisplayed()`.
- `testBenefitIconsAltAttributes` проверяет атрибут `alt` у каждой иконки на наличие и содержание, которое должно соответствовать ключевым словам `"Customization", "Interface", "Platform", "Base"`.

4.3.2 Проверка текстов под иконками

- `testBenefitTextsCount` проверяет наличие 4 элементов с классом `benefit-txt`.
- `testBenefitTextsContent` сравнивает текст каждого блока с ожидаемым значением (с учётом переносов строк `\n`), а также проверяет наличие ключевых слов (`"EPAM", "flexible", "multiplatform"`) в соответствующих блоках.
- `testBenefitTextsClasses` проверяет, что каждый текстовый блок имеет атрибут `class`, содержащий `"benefit-txt"` и не скрыт стилем (`display != "none"`).

4.3.3 Проверка iframe с кнопкой

- `testFrameExistence` проверяет наличие `iframe` с `id="frame"`, его видимость и атрибут `src`, содержащий `"jdi-testing"`.
- `testFrameContent` переключает контекст драйвера в `iframe`, находит кнопку с `id="frame-button"` и проверяет её видимость, текст и активность, после этого возвращается к основному контенту.

4.3.4 Проверка левого меню (Sidebar)

- `testLeftMenuCount` проверяет наличие 5 пунктов меню с помощью селектора `"ul.sidebar-menu.left > li"`.
- `testLeftMenuTexts` сравнивает текст каждого пункта меню с ожидаемыми значениями: `"Home", "Contact form", "Service", "Metals & Colors", "Elements packs"`.

4.4 Результаты тестирования

В ходе выполнения второй части работы была успешно проведена проверка отображения компонентов пользовательского интерфейса и их взаимодействия. С помощью Selenium WebDriver и TestNG было обеспечено автоматизированное управление браузером Chrome в процессе выполнения тестов, а также получен доступ к частям интерфейса по ID, CSS селекторам и по имени класса.

Применение механизма «мягких утверждений» позволило выполнить комплексную проверку пользовательского интерфейса сайта в рамках одного запуска тестового метода `HomePageContentTests`. Для обеспечения точных результатов выполнено управление сессией браузера: открытие браузера, авторизация, по завершении всех тестов - автоматическое закрытие браузера.

Протестированы различные элементы страницы: текст заголовка страницы, наличие input-полей, textarea, кнопки в форме, наличие и видимость иконок; проверка текстов под иконками, iframe с кнопкой и левого меню.

Результаты тестирования представлены на рисунке 8.

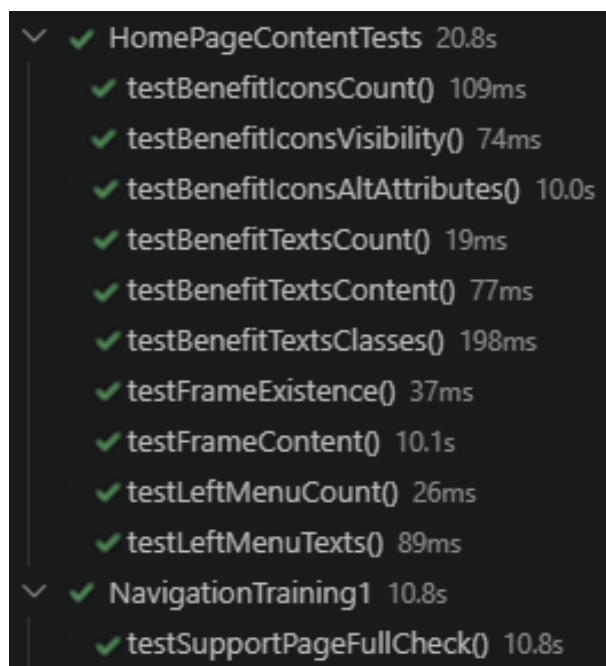


Рис. 8. Результаты тестирования

Заключение

В ходе выполнения данной лабораторной работы были изучены основные аспекты автоматизированного тестирования программного обеспечения. Были получены навыки написания и выполнения юнит-тестов с использованием фреймворка JUnit 5 для Java библиотеки lab-1.0.jar. В результате проведённого тестирования четырех функций библиотеки было найдено два дефекта, в которых поведение тестируемых функций отличалось от ожидаемой.

Также были получены навыки автоматизации тестирования веб-страниц с помощью WebDriver и TestNG. Разработаны два тестовых сценария, проверяющих корректность отображения страниц заданного сайта и взаимодействия с компонентами пользовательского интерфейса. Все тесты были выполнены успешно, что подтверждает корректность работы тестируемой веб-страницы.

В результате выполненной работы можно сделать вывод, что оба метода тестирования, ручной и автоматизированный, способны помочь в нахождении ошибок. Автоматизированное тестирование обладает большей производительностью по сравнению с ручным, однако ручное тестирование позволяет выявить неожиданные проблемы, не рассмотренные при автоматизации. Таким образом, лучше использовать оба метода вместе в процессе тестирования.

Освоенные инструменты (JUnit 5, Selenium WebDriver, TestNG) могут быть эффективно применены в реальных проектах для автоматизации рутинных проверок и существенного сокращения времени тестирования.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.
- [2] Курочкин, М.А., Глазунов, В.В. Основы тестирования программного обеспечения - Санкт-Петербург: 2024 - 151 с.

Приложение 1. Исходный код ArrayUtilsTests

Листинг 1. Исходный код ArrayUtilsTests

```
1 package ru.spbstu.hsai;
2 import org.junit.jupiter.api.BeforeAll;
3
4 public abstract class ArrayUtilsTest {
5     protected static final double DELTA = 1e-4;
6     @BeforeAll
7     public static void setUp() {
8         System.out.println("Initializing shared test array before tests");
9     }
10 }
```

Приложение 2. Исходный код AverageTest

Листинг 2. Исходный код AverageTest

```
1 package ru.spbstu.hsai;
2 import org.junit.jupiter.api.DisplayName;
3 import org.junit.jupiter.params.ParameterizedTest;
4 import org.junit.jupiter.params.provider.Arguments;
5 import org.junit.jupiter.params.provider.MethodSource;
6 import static org.junit.jupiter.api.Assertions.*;
7 import java.util.stream.Stream;
8
9 @DisplayName("ArrayUtils.average tests")
10 public class AverageTest extends ArrayUtilsTest {
11     private static Stream<Arguments> validAverageCases() {
12         return Stream.of(
13             Arguments.of(new int[]{5, 7, 23, -2, 4}, 7.4, "Some digits"),
14             Arguments.of(new int[]{Integer.MIN_VALUE}, (double)Integer.MIN_VALUE, "MIN VALUE"),
15             Arguments.of(new int[]{Integer.MAX_VALUE}, (double)Integer.MAX_VALUE, "MAX VALUE")
16         );
17     }
18
19     private static Stream<Arguments> invalidAverageCases() {
20         return Stream.of(
21             Arguments.of((int[]) null, "null array"),
22             Arguments.of(new int[]{}, "empty array")
23         );
24     }
25
26     @ParameterizedTest
27     @MethodSource("validAverageCases")
28     void testAverageValid(int[] input, double expected, String description) {
29         double result = ArrayUtils.average(input);
30         assertEquals(expected, result, DELTA, "Failed: " + description);
31     }
```

```

32 }
33
34 @ParameterizedTest
35 @MethodSource("invalidAverageCases")
36 void testAverageInvalid(int[] input, String description) {
37     assertThrows(IllegalArgumentException.class,
38         () -> ArrayUtils.average(input),
39         "Expected exception for: " + description);
40 }
41 }

```

Приложение 3. Исходный код CountOccurrencesTest

Листинг 3. Исходный код CountOccurrencesTest

```

1 package ru.spbstu.hsai;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.params.ParameterizedTest;
5 import org.junit.jupiter.params.provider.Arguments;
6 import org.junit.jupiter.params.provider.MethodSource;
7 import org.junit.jupiter.params.provider.ValueSource;
8 import static org.junit.jupiter.api.Assertions.*;
9
10 import java.util.stream.Stream;
11
12 @DisplayName("ArrayUtils.countOccurrences tests")
13 public class CountOccurrencesTest extends ArrayUtilsTest {
14
15     private static Stream<Arguments> validCountCases() {
16         return Stream.of(
17             Arguments.of(new int[]{7}, 7, 1, "Single element [7], value=7"),
18             Arguments.of(new int[]{1, 2, 3}, 1, 1, "Value at first position [1,2,3], search=1"),
19             Arguments.of(new int[]{1, 1, 2}, 1, 2, "Value at first+second [1,1,2], search=1"),
20             Arguments.of(new int[]{2, 3, 1}, 1, 1, "Value at last position [2,3,1], search=1"),
21             Arguments.of(new int[]{1, 2, 1}, 1, 2, "Value at first+last [1,2,1], search=1"),
22             Arguments.of(new int[]{2, 3, 3, 3}, 3, 3, "Multiple occurrences [2,3,3,3], search=3"),
23             Arguments.of(new int[]{1, 2, 3}, 99, 0, "Absent value [1,2,3], search=99"),
24             Arguments.of(new int[]{}, 5, 0, "Empty array, any value")
25         );
26     }
27
28     private static Stream<Arguments> invalidCountCases() {
29         return Stream.of(
30             Arguments.of((int[]) null, 5, "null array"),
31             Arguments.of((int[]) null, 0, "null array with zero value")
32         );
33     }
34 }

```



```

33 }
34
35 @ParameterizedTest
36 @MethodSource("validCountCases")
37 @DisplayName("Count occurrences: valid cases")
38 void testCountOccurrencesValid(int[] array, int value, int expected,
    String description) {
39     int result = ArrayUtils.countOccurrences(array, value);
40     assertEquals(expected, result, "Failed for: " + description);
41 }
42
43 @ParameterizedTest
44 @MethodSource("invalidCountCases")
45 @DisplayName("Count occurrences: invalid input throws exception")
46 void testCountOccurrencesInvalid(int[] array, int value, String
    description) {
47     assertThrows(IllegalArgumentException.class,
48         () -> ArrayUtils.countOccurrences(array, value),
49         "Expected exception for: " + description);
50 }
51
52 @ParameterizedTest
53 @MethodSource("boundaryCases")
54 @DisplayName("Count occurrences: boundary conditions")
55 void testCountOccurrencesBoundary(int[] array, int value, int expected
    , String description) {
56     int result = ArrayUtils.countOccurrences(array, value);
57     assertEquals(expected, result, "Boundary failed: " + description);
58 }
59
60 private static Stream<Arguments> boundaryCases() {
61     return Stream.of(
62         Arguments.of(new int[]{0, Integer.MIN_VALUE, Integer.MIN_VALUE},
63             Integer.MIN_VALUE, 2,
64             "MIN_VALUE occurrences"),
65         Arguments.of(new int[]{0, Integer.MAX_VALUE, Integer.MAX_VALUE},
66             Integer.MAX_VALUE, 2,
67             "MAX_VALUE occurrences")
68     );
69 }

```

Приложение 4. Исходный код ReverseTest

Листинг 4. Исходный код ReverseTest

```

1 package ru.spbstu.hsai;
2 import org.junit.jupiter.api.DisplayName;
3 import org.junit.jupiter.params.ParameterizedTest;
4 import org.junit.jupiter.params.provider.Arguments;
5 import org.junit.jupiter.params.provider.MethodSource;
6 import org.junit.jupiter.params.provider.ValueSource;
7 import static org.junit.jupiter.api.Assertions.*;

```

```

8 import java.util.stream.Stream;
9
10 @DisplayName("ArrayUtils.reverse tests")
11 public class ReverseTest extends ArrayUtilsTest {
12
13     private static Stream<Arguments> validReverseCases() {
14         return Stream.of(
15             Arguments.of(new int[]{5}, new int[]{5}, "Single element [5]"),
16             Arguments.of(
17                 new int[]{1, -2, 3, 0, -5},
18                 new int[]{-5, 0, 3, -2, 1},
19                 "Five different [1,-2,3,0,-5]"
20             ),
21             Arguments.of(
22                 new int[]{Integer.MAX_VALUE},
23                 new int[]{Integer.MAX_VALUE},
24                 "Single element [MAX_VALUE]"
25             ),
26             Arguments.of(
27                 new int[]{Integer.MIN_VALUE},
28                 new int[]{Integer.MIN_VALUE},
29                 "Single element [MIN_VALUE]"
30             ),
31             Arguments.of(new int[]{}, new int[]{}, "Empty array []")
32         );
33     }
34
35     private static Stream<Arguments> invalidReverseCases() {
36         return Stream.of(
37             Arguments.of((int[]) null, "null array")
38         );
39     }
40
41     @ParameterizedTest
42     @MethodSource("validReverseCases")
43     @DisplayName("Reverse: valid cases")
44     void testReverseValid(int[] input, int[] expected, String description)
45     {
46         int[] result = ArrayUtils.reverse(input);
47         assertEquals(expected, result, "Failed for: " + description);
48     }
49
50     @ParameterizedTest
51     @MethodSource("invalidReverseCases")
52     @DisplayName("Reverse: null array throws exception")
53     void testReverseInvalid(int[] input, String description) {
54         assertThrows(IllegalArgumentException.class,
55             () -> ArrayUtils.reverse(input),
56             "Expected exception for: " + description);
57     }
58 }

```

Приложение 5. Исходный код SumTest

Листинг 5. Исходный код SumTest

```
1 package ru.spbstu.hsai;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.params.ParameterizedTest;
5 import org.junit.jupiter.params.provider.Arguments;
6 import org.junit.jupiter.params.provider.MethodSource;
7 import static org.junit.jupiter.api.Assertions.*;
8
9 import java.util.stream.Stream;
10
11 @DisplayName("ArrayUtils.sum tests")
12 public class SumTest extends ArrayUtilsTest {
13
14     private static Stream<Arguments> validSumCases() {
15         return Stream.of(
16             Arguments.of(new int[]{5}, 5, "Single element [5]"),
17             Arguments.of(new int[]{1, -2, 3, 0, -5, 3, 1}, 1, "Mixed
18 [1,-2,3,0,-5,3,1]"),
19             Arguments.of(new int[]{Integer.MIN_VALUE, 1}, Integer.MIN_VALUE + 1,
20 "MIN_VALUE + 1"),
21             Arguments.of(new int[]{Integer.MAX_VALUE, -1}, Integer.MAX_VALUE -
22 1, "MAX_VALUE - 1"),
23             Arguments.of(new int[]{}, 0, "Empty array []")
24         );
25     }
26
27     private static Stream<Arguments> invalidSumCases() {
28         return Stream.of(
29             Arguments.of((int[]) null, "null array")
30         );
31     }
32
33     @ParameterizedTest
34     @MethodSource("validSumCases")
35     @DisplayName("Sum: valid cases")
36     void testSumValid(int[] input, int expected, String description) {
37         int result = ArrayUtils.sum(input);
38         assertEquals(expected, result, "Failed for: " + description);
39     }
40
41     @ParameterizedTest
42     @MethodSource("invalidSumCases")
43     @DisplayName("Sum: null array throws exception")
44     void testSumInvalid(int[] input, String description) {
45         assertThrows(IllegalArgumentException.class,
46             () -> ArrayUtils.sum(input),
47             "Expected exception for: " + description);
48     }
49 }
```

Приложение 6. Исходный код NavigationTraining1

Листинг 6. Исходный код NavigationTraining1

```
1 package ru.spbstu.hsai;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.OutputType;
5 import org.openqa.selenium.TakesScreenshot;
6 import org.openqa.selenium.WebElement;
7 import org.openqa.selenium.support.ui.ExpectedConditions;
8 import org.openqa.selenium.support.ui.WebDriverWait;
9 import org.testng.annotations.Test;
10 import org.testng.asserts.SoftAssert;
11
12 import java.io.File;
13 import java.io.FileWriter;
14 import java.io.IOException;
15 import java.time.Duration;
16 import java.util.List;
17
18 public class NavigationTraining1 extends DriverSetup {
19
20     // Метод навигации с улучшенной стабильностью
21     private void navigateTo(String menuItem) {
22         WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds
23             (10));
24
25         try {
26             // 1. Клик по Service (dropdown)
27             WebElement serviceBtn = wait.until(ExpectedConditions.
28                 elementToBeClickable(
29                     By.xpath("//a[contains(@class,'dropdown-toggle') and normalize-
30                         space()='Service']")
31                 ));
32             serviceBtn.click();
33
34             // 2. Клик по Support
35             WebElement target = wait.until(ExpectedConditions.
36                 elementToBeClickable(
37                     By.xpath("//ul[contains(@class,'dropdown-menu')]/a[normalize-
38                         space()=' " + menuItem + "']")
39                 ));
40             target.click();
41
42             // 3. Ждём переход
43             wait.until(ExpectedConditions.urlContains("support"));
44
45         } catch (Exception e) {
46             throw new RuntimeException("Не удалось перейти в меню: " + menuItem,
47                 e);
48         }
49     }
50
51     // Утилита для сохранения страницы при ошибке
52     private void saveDebugInfo(String testName, Exception e) {
```

```

47     try {
48         // Скриншот
49         File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.
FILE);
50         src.renameTo(new File("error_" + testName + ".png"));
51
52         // HTML исходник
53         String html = driver.getPageSource();
54         FileWriter writer = new FileWriter("error_" + testName + ".html");
55         writer.write(html);
56         writer.close();
57
58     } catch (IOException ioException) {
59         System.err.println("Failed to save debug info: " + ioException.
getMessage());
60     }
61 }
62
63
64 @Test
65 public void testSupportPageFullCheck() {
66     SoftAssert softAssert = new SoftAssert();
67     String testName = "testSupportPageFullCheck";
68
69     try{
70         // 1. Navigate to the "Support" page using the main menu
navigation
71         navigateTo("Support");
72
73         // 2. Assert that the page title contains the word "support" (case
-insensitive)
74         String pageTitle = driver.getTitle().toLowerCase();
75         System.out.println(pageTitle);
76         softAssert.assertTrue(
77             pageTitle.contains("support"), "Page title does not contain '
support'. In reality: " + pageTitle);
78
79         // 3. Assert that a form or contact block is present on the page
80         List<WebElement> forms = driver.findElements(By.tagName("form"));
81         WebElement target = null;
82         // Если форма найдена, используем её, иначе ищем блок с контактами
83         if (!forms.isEmpty()) {
84             target = forms.get(0);
85         } else {
86             List<WebElement> contactBlocks = driver.findElements(
87                 By.cssSelector("div[class*='contact'], section[class*='contact']
")
88             );
89             softAssert.assertFalse(
90                 contactBlocks.isEmpty(),
91                 "Neither form nor contact block found"
92             );
93
94             if (!contactBlocks.isEmpty()) {
95                 target = contactBlocks.get(0);
96             }

```

```

97     }
98     // 4. Assert that the form is displayed and visible to the user
99     if (target != null) {
100         softAssert.assertTrue(
101             !target.isDisplayed(),
102             "Form/contact block is not visible"
103         );
104
105         // 5. Assert that the form contains at least one input field (<
106         input>)
107         List<WebElement> inputs = target.findElements(By.tagName("input"
108         ));
109         softAssert.assertTrue(
110             inputs.size() >= 1,
111             "No <input> fields inside form"
112         );
113
114         // 6. Assert that the form contains at least one textarea
115         element (<textarea>)
116         List<WebElement> textAreas = target.findElements(By.tagName("
117         textarea"));
118         softAssert.assertTrue(
119             textAreas.isEmpty(),
120             "No <textarea> inside form"
121         );
122
123         // 7. Assert that the form contains a submit button (<button> or
124         <input type="submit">)
125         List<WebElement> buttons = target.findElements(By.cssSelector(
126             "button, input[type='submit'], input[type='button']"
127         ));
128         softAssert.assertFalse(
129             buttons.isEmpty(),
130             "No submit button inside form"
131         );
132     }
133 } catch (Exception e) {
134     saveDebugInfo(testName, e);
135     softAssert.fail("Test " + testName + " failed with exception: " +
136     e.getMessage());
137 } finally {
138     softAssert.assertAll();
139 }
140 }
141 }

```

Приложение 7. Исходный код HomePageContentTests

Листинг 7. Исходный код HomePageContentTests

```

1 package ru.spbstu.hsai;
2

```

```

3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebElement;
5 import org.testng.annotations.Test;
6 import java.util.List;
7 import static org.testng.Assert.*;
8
9 public class HomePageContentTests extends DriverSetup {
10
11     // 1. Проверка количества иконок преимуществ (Benefit Icons)
12     @Test(priority = 1)
13     public void testBenefitIconsCount() {
14         List<WebElement> icons = driver.findElements(By.cssSelector(".
15         benefit-icon span, .benefit-icon img, .benefit-item img"));
16         assertEquals(icons.size(), 4, "Expected 4 benefit icons, found: " +
17         icons.size());
18     }
19
20     // 2. Проверка видимости всех иконок
21     @Test(priority = 2, dependsOnMethods = "testBenefitIconsCount")
22     public void testBenefitIconsVisibility() {
23         List<WebElement> icons = driver.findElements(By.cssSelector(".
24         benefit-icon span, .benefit-icon img, .benefit-item img"));
25         for (int i = 0; i < icons.size(); i++) {
26             assertTrue(icons.get(i).isDisplayed(), "Benefit icon #" + (i+1) +
27             " is not displayed");
28         }
29     }
30
31     // 3. Проверка атрибутов alt у иконок доступность()
32     @Test(priority = 3, dependsOnMethods = "testBenefitIconsVisibility")
33     public void testBenefitIconsAltAttributes() {
34         List<WebElement> icons = driver.findElements(By.cssSelector(".
35         benefit-item img"));
36         String[] expectedStrings = {"Customization", "Interface", "Platform",
37         "Base"};
38         for (int i = 0; i < Math.min(icons.size(), expectedStrings.length);
39         i++) {
40             String altText = icons.get(i).getAttribute("alt");
41             assertNotNull(altText, "Icon #" + (i+1) + " is missing alt
42             attribute");
43             assertFalse(altText.trim().isEmpty(), "Icon #" + (i+1) + " has
44             empty alt attribute");
45             assertTrue(altText.toLowerCase().contains(expectedStrings[i].
46             toLowerCase()),
47             "Icon #" + (i+1) + " alt attribute does not contain expected text.
48             Expected: " +
49             expectedStrings[i] + ", but got: " + altText);
50         }
51     }
52
53     // 4. Проверка количества текстовых блоков под иконками
54     @Test(priority = 4)
55     public void testBenefitTextsCount() {
56         List<WebElement> texts = driver.findElements(By.className("benefit-
57         txt"));
58     }
59 }

```

```

46     assertEquals(texts.size(), 4, "Expected 4 benefit text blocks, found
47 : " + texts.size());
48 }
49 // 5. Проверка содержания текстов преимуществ
50 @Test(priority = 5, dependsOnMethods = "testBenefitTextsCount")
51 public void testBenefitTextsContent() {
52     List<WebElement> texts = driver.findElements(By.className("benefit-
53 txt"));
54     String[] expectedTexts = {
55         "To include good practices\nand ideas from successful\nEPAM
56 project",
57         "To be flexible and\ncustomizable",
58         "To be multiplatform",
59         "Already have good base\n(about 20 internal and\nsome external
60 projects),\nwish to get ...more"
61     };
62     for (int i = 0; i < texts.size(); i++) {
63         String actualText = texts.get(i).getText().trim();
64         assertFalse(actualText.isEmpty(), "Benefit text #" + (i+1) + " is
65 empty");
66         assertEquals(actualText, expectedTexts[i],
67             "Benefit text #" + (i+1) + " does not match expected. Expected: "
68 +
69             expectedTexts[i] + ", but got: " + actualText);
70         if (i == 0) {
71             assertTrue(actualText.contains("EPAM"), "Benefit text #1 does
72 not contain 'EPAM'");
73         } else if (i == 1) {
74             assertTrue(actualText.toLowerCase().contains("flexible"), "
75 Benefit text #2 does not contain 'flexible'");
76         } else if (i == 2) {
77             assertTrue(actualText.toLowerCase().contains("multiplatform"), "
78 Benefit text #3 does not contain 'multiplatform'");
79         }
80     }
81 }
82 // 6. Проверка классов и стилей текстовых блоков
83 @Test(priority = 6, dependsOnMethods = "testBenefitTextsContent")
84 public void testBenefitTextsClasses() {
85     List<WebElement> texts = driver.findElements(By.className("benefit-
86 txt"));
87     for (int i = 0; i < texts.size(); i++) {
88         String className = texts.get(i).getAttribute("class");
89         assertNotNull(className, "Benefit text block #" + (i+1) + " is
90 missing class attribute");
91         assertTrue(className.contains("benefit-txt"),
92             "Benefit text block #" + (i+1) + " does not contain 'benefit-txt'
93 in class attribute");
94
95         String displayValue = texts.get(i).getCssValue("display");
96         assertFalse(displayValue.equalsIgnoreCase("none"),
97             "Benefit text block #" + (i+1) + " is hidden (display: none)");
98     }
99 }

```



```

90 // 7. Проверка наличия iframe с кнопкой
91 @Test(priority = 7)
92 public void testFrameExistence() {
93     List<WebElement> iframes = driver.findElements(By.id("frame"));
94     assertFalse(iframes.isEmpty(), "Iframe with id 'frame' not found");
95     String src = iframes.get(0).getAttribute("src");
96     assertNotNull(src, "Iframe src attribute is null");
97     assertFalse(src.trim().isEmpty(), "Iframe src attribute is empty");
98     assertTrue(src.contains("jdi-testing"), "Iframe src does not contain
99     'jdi-testing'");
100 }
101
102 // 8. Проверка содержимого iframe переключение( и проверка кнопки)
103 @Test(priority = 8, dependsOnMethods = "testFrameExistence")
104 public void testFrameContent() {
105     driver.switchTo().frame("frame");
106     try{
107         WebElement frameButton = driver.findElement(By.id("frame-button"))
108     ;
109         assertTrue(frameButton.isDisplayed(), "Button with id 'frame-
110         button' is not displayed in iframe");
111         String buttonText = frameButton.getText().trim();
112         assertEquals(buttonText, "Frame Button", "Button text does not
113         match expected 'Frame Button'");
114         assertTrue(frameButton.isEnabled(), "Button with id 'frame-button'
115         is not enabled in iframe");
116     } finally {
117         driver.switchTo().defaultContent()
118         driver.switchTo().defaultContent();
119     }
120 }
121
122 // 9. Проверка количества элементов левого меню (Sidebar)
123 @Test(priority = 9)
124 public void testLeftMenuCount() {
125     List<WebElement> menuItems = driver.findElements(By.cssSelector("ul.
126     sidebar-menu.left > li"));
127     assertEquals(5, menuItems.size(), "Expected 5 menu items in the left
128     sidebar");
129 }
130
131 // 10. Проверка текстов левого меню
132 @Test(priority = 10, dependsOnMethods = "testLeftMenuCount")
133 public void testLeftMenuTexts() {
134     List<WebElement> menuItems = driver.findElements(By.cssSelector("ul.
135     sidebar-menu.left > li"));
136     String[] expectedTexts = {"Home", "Contact form", "Service", "Metals
137     & Colors", "Elements packs"};
138     for (int i = 0; i < menuItems.size(); i++) {
139         String actualText = menuItems.get(i).getText().trim();
140         assertEquals(actualText, expectedTexts[i],
141             "Menu item #" + (i+1) + " text does not match expected. Expected:
142             '" +
143             expectedTexts[i] + "', but got: '" + actualText + "'");
144     }
145 }

```

136	}
137	}